Software Architecture for Essential and Accidental Uncertainty

George Rudolph

Neil Harrison, Peter Aldous

Utah Valley University

1

presented at HICSS-59

Introduction: A bit of History

The ENIAC: Electronic Numerical Integrator and Computer

First general-purpose computer to solve “a large class of numerical problems”

problems were precisely defined

results could be deterministically computed

Progress: More Applications

Business Applications

Variation in amount and nature of input data

Data Communication

Data transmission issues: timing, failure, corruption,...

Control Systems

Must handle all different problems reliably/robustly

Artificial Intelligence

History: Summary

- Original problems solved by computers were mathematical
  - _precisely _ *specified*
  - _precisely _ *defined outputs*
  - _precisely _ *defined algorithms*
  - even if you didn't know the exact answer yet
- As software capabilities increase
  - the inputs, outputs, and even processing

Current State of the Art

- 2010 David Garlan: uncertainty should be a first-class concern in software design and development
  - He was late to the party...
- Uncertainty research and practice is ENTIRELY focused on
  - fault tolerance and
  - similar approaches

What This Misses

__misses __ __a large part of __ **uncertainty**

demands an in-depth investigation of the nature of uncertainty

Yet more history

Essential and Accidental Uncertainty

Accidental uncertainty : Uncertainty that can be separated from the problem at hand. There exists a transformation that removes the uncertainty.

Essential uncertainty : Uncertainty that must be addressed as part of the problem. No transformation can exist that removes the uncertainty from the problem.

Uncertainty and Data

- Uncertainty can be characterized by considering the **data** of a system
    - After all, computing is processing of data
- Consider the following
    - Uncertainty of the input (can be essential or accidental)
    - Uncertainty of the output (generally essential)

Input: Uncertainty Examples

- Unreliable data transmission:
    - Accidental, because not part of the essence of the data. Datacomm protocols implement transformations to remove the uncertainty
- Amount of data being processed:
    - Often accidental, because we can often handle it in subsets
- User input errors

Input Uncertainty

Other cases of input uncertainty exist that are essential.

We will discuss these later

Uncertainty in the Transformation

- By itself, it is generally kind of accidental.
- Examples:
  - Hardware glitches during processing: accidental, approaches include duplication of hardware
  - Different compilers produce different programs that do the same thing: we generally don’t care if

Output Uncertainty

Accidental uncertainty of output is generally not very interesting

Essential output uncertainty spans a wide spectrum of the nature of the uncertainty

Requires a corresponding spectrum of architectural and design approaches

Essential Output Uncertainties

Output Characteristics	Example
The right answer(s) can be known	Sales Website (how well did a product sell?)
	10^10^48 to 10^10^170 possible games)
Unknown right answer, but can guess	Image recognition

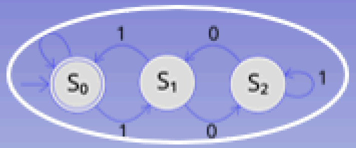
More …

Output Characteristics	Example
The effect is unknown; the system is chaotic	Weather forecasting
No single right answer, and the problem is vague	Automated prose writing

Thought Exercise

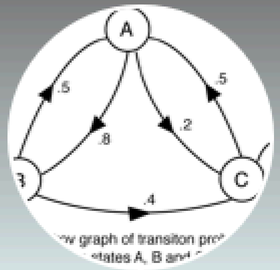
- Q: How would you characterize automatic software coding?
- A: it's a trick question. It depends on the nature of the software being written.
- Q: Can you precisely specify what the generated software should do?
    - Point: If not , how in the world can you verify it???

Spectrum of Uncertainty



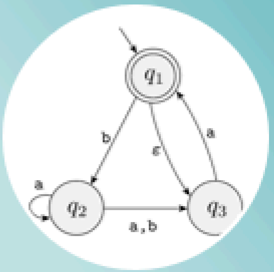
Deterministic

- Exactly one action possible for an input from a given program state
- Always the same action for input, state pairs
- No uncertainty



Probabilistic

- More than one action possible for an input from a given program state
- Each transition has a known probability
- Probabilities out of each possible state add to 1
- Some uncertainty



Nondeterministic

- More than one action possible for an input from a given program state
- We don't know the transition probabilities
- And/or cannot easily determine possible probabilities
- High uncertainty

Less Uncertainty ← → More Uncertainty

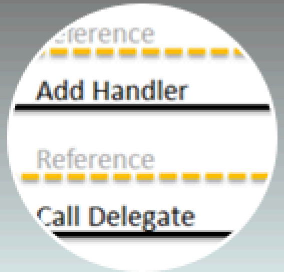
Architectural Approaches

- Different levels and characteristics of uncertainty suggest different architectural approaches, including patterns, styles, and tactics
- Virtually no uncertainty: Batch processing
- Pipes and Filters, Layers, Broker: well-suited for certain types of accidental uncertainty
- Greater essential uncertainty leads to AI-based approaches
    - Blackboard

Spectrum of Architectural Approaches

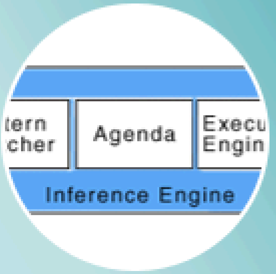
Procedural Code

- Fixed order
- Know algorithm
- Know data
- Can write code
- No uncertainty



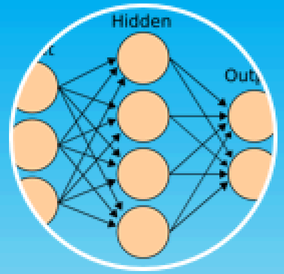
Event Driven

- Order of execution varies
- Key events are known
- What to do is known



Rules Engine

- Know Key Features
- Can write rules
- Order of execution is uncertain
- Rules may change over time



Neural Net

- Have lots of data
- Cannot write rules
- Do not yet know key features
- Looking for patterns in data

Less Uncertainty ← → More Uncertainty

An Architecture Process for Uncertainty

- Identify the uncertainties associated with the system
- For each, determine if it is essential or accidental
  - Study the source of the uncertainty (external is usually accidental)
  - Determine if there is a transformation to remove the uncertainty

3. Analyze Uncertainties

Analysis of the Input Data:

It is because of imprecise/incomplete specification?

It is because if variability in the data (such as transmission, user input errors, etc.)?

What can and cannot be known about the data at runtime?

Analysis of the Output Data

What is the objective of the system? How can we know if it meets its objective?

Is there a known correct output?

Is there a correct output, but we cannot know if it is correct?

Can the output be correctly and deterministically derived from the inputs?

Are there multiple correct outputs?

If there are multiple correct outputs, is one more preferred than others?

Analysis of Transformation from Input to Output

Is the relationship between input and output deterministic?

If it is deterministic, is it tractable?

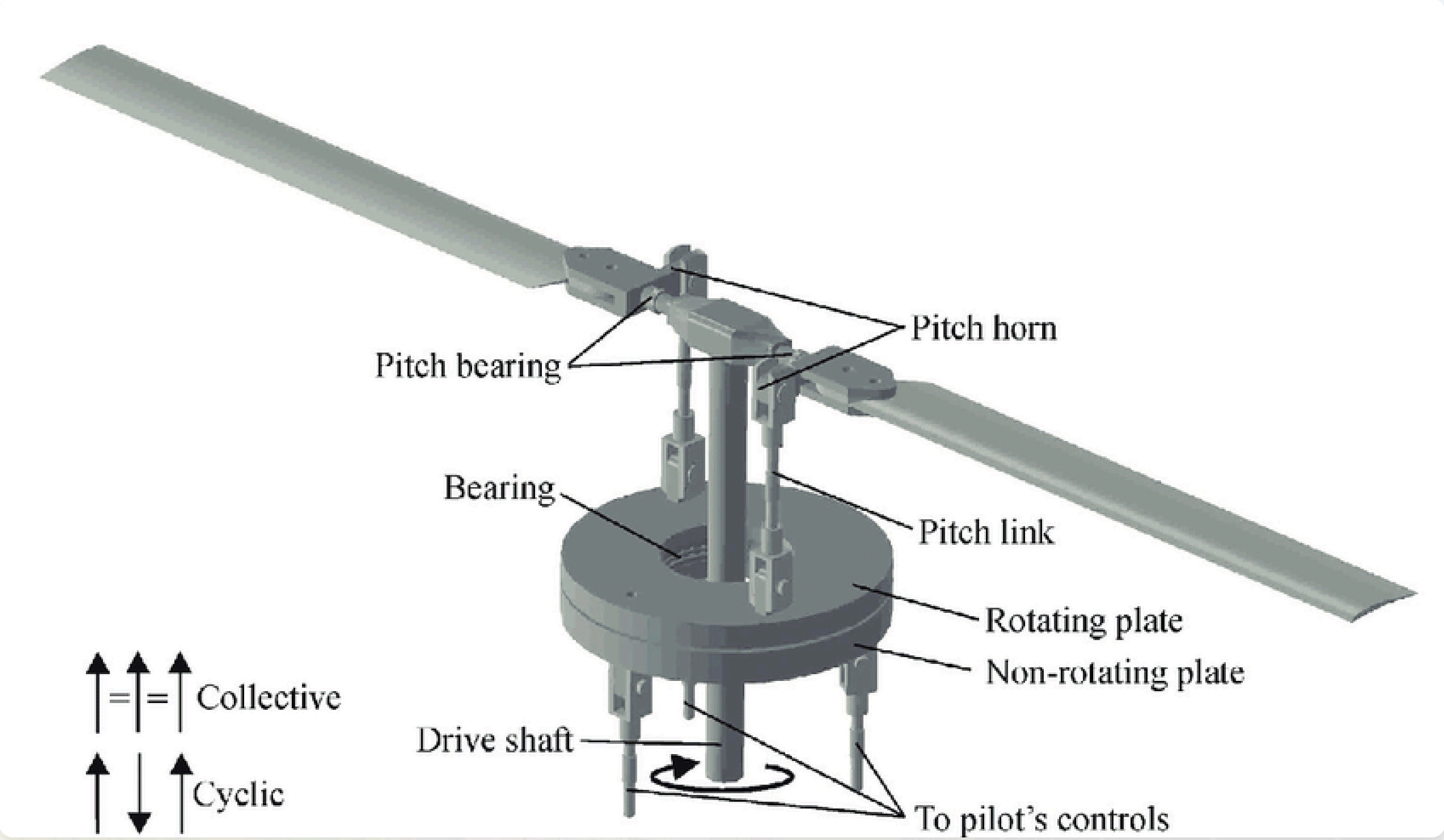
Case Study (Real Life)

Helicopter Rotor Balancing

If the main rotor of a helicopter is not perfectly balanced, it causes excess vibration

Vibration causes excess wear, loss of power, and even compromised safety

The main rotor is a complex system, with several different adjustment points.


Pitch horn
Pitch bearing
Bearing
Pitch link
Rotating plate
Non-rotating plate
Collective
Drive shaft
Cyclic
To pilot's controls

Uncertainty Analysis: Input

- Input is known, and precisely specified:
  - Vibration of the rotor can be precisely measured
  - Positions of blades and adjustment points
  - History of adjustments/response (!) (was missed!)
- Uncertainty: possible user error in entering data
  - External: consider standard FT techniques
- Uncertainty: each helicopter has unique wear patterns

Uncertainty Analysis: Output

- Overall objective known and can be measured: how much did the vibration change?
- Desired output is a range: lower vibration is better, of course.
  - Standard of acceptable vibration (0.5 IPS, inches per second)
- Single solution? There may be different sets or sequences of adjustments to achieve the goal.

Uncertainty Analysis: Processing

- Is the relationship between input and output deterministic?
  - Unknown!
  - Combinations of adjustments did not appear to deterministically result in reduced vibration
  - System appeared to be chaotic .
- Conclusion:

However:

The analysis had not fully considered all the aspects of the input, output, and processing!

Individual differences of each helicopter

History of adjustments and responses

Further Analysis Revealed...

- Each helicopter has a unique wear profile
  - Neural network approach was based on many samples from many different helicopters
  - Result: generic approximation
- The adjustment/response history reveals the unique profile
- There is a sequence of adjustments that is deterministic and repeatable
  - Employs feedbac

Results

System can reliably balance main rotors to 0.05 IPS, 10x better than the standard.

System is actively being marketed to US Army and other governmental and private organizations.

Demonstrates the importance and effectiveness of understanding the essential and accidental uncertainty of a system

Case Study: Online Bookstore

- Initially:
- Input and output analysis:
  - Input and output well specified.
  - Numerous accidental uncertainties, such as unreliability of the network.
- Straightforward

Case Study: Bookstore evolution

- Add a "related books" feature
  - Which books are related to the selected book?
  - An essential uncertainty
  - Output cannot be known beforehand
  - Not one single right answer
  - Approaches are [illegible] but are

- Examine browsing history to promote products
    - Even more uncertainty
    - Problem is less well defined
    - Consider machine learning
- See uncertainty spectrum as discussed earlier

References

Harrison, N.B., Rudolph, G., and Aldous, P. Software Architecture for Essential and Accidental Uncertainty, Hawaii International Conference on the System Sciences (HICSS), 2026.

Harrison, N.B., Rudolph, G., and Aldous, P. Is Artificial Intelligence the Answer to Everything?, 2025 Intermountain Conference on Engineering, Technology and Computing (IETC), 2025.

Thank you!

Questions?